

BÀI TẬP VỀ NHÀ — Ôn tập Bài 13 → 16

Phạm vi ôn tập: Giai đoạn 4 — Tính năng nâng cao & Kiến trúc (Two-way binding, Services & DI, Routing, Reactive Forms)

Nguyên tắc quan trọng: KHÔNG động vào project Smart Recipe Box. Toàn bộ bài tập làm trên project riêng `bai-tap-tra-sua`.

Chuẩn bị

Nếu bạn ĐÃ làm bài tập 7–12

Dùng luôn project cũ, bỏ qua phần code khởi đầu bên dưới.

Nếu bạn CHƯA có project

```
cd <thư mục chứa project của bạn>
```

```
ng new bai-tap-tra-sua --style=css --ssr=false --routing=true --skip-tests
```

⚠ Lần này **cần** `--routing=true` vì Nhiệm vụ C15 là về routing. Cờ này tạo sẵn file `app.routes.ts` và thêm `provideRouter` vào `app.config.ts`.

```
cd bai-tap-tra-sua && ng serve --port 4300
```

💡 Nếu cổng 4300 bị chiếm, dùng cổng khác: `ng serve --port 4301`. Cách kiểm tra cổng nào đang bị chiếm: `lsof -ti:4300`

Code khởi đầu (trạng thái tương đương "đã xong Bài 12")

🎨 **Bộ bài này CÓ chấm CSS.** Code khởi đầu để trơn hoàn toàn — **không có một class nào**. Việc đặt tên class và viết CSS là phần việc của bạn, và nó được tính điểm theo **ba tiêu chí của Bài 10**: có dùng Flexbox, có thứ bậc thị giác rõ ràng, có khoảng trắng hợp lý.

Mỗi nhiệm vụ C13–C16 đều có mục 🎨 **Yêu cầu CSS** riêng, cộng thêm một nhiệm vụ **C-CSS** tổng kết ở cuối. Bảng màu có sẵn ở đó nếu bạn không muốn tự chọn màu.

- ▶  `src/app/models.ts`
- ▶  `src/app/mock-drinks.ts`
- ▶  `src/app/app.ts` + `app.html`
- ▶  `src/app/drink-list/drink-list.ts` + `drink-list.html`
- ▶  `src/app/drink-detail/drink-detail.ts` + `drink-detail.html`

Chạy `ng serve --port 4300`, thấy 3 nút chọn món và phần chi tiết là bạn đã sẵn sàng.



PHẦN A — Lý thuyết

Trả lời ngắn gọn bằng lời của bạn.

1. Vì sao với signal ta phải dùng dạng tách rời `[ngModel]` + `(ngModelChange)` mà không dùng được cú pháp gộp `[(ngModel)]`?
2. `$event` trong `(ngModelChange)="keyword.set($event)"` là gì?
3. Phân biệt `.map()`, `.filter()` và `.find()`: mỗi cái trả về gì?
4. Vì sao khi tìm kiếm nên gọi `.toLowerCase()` ở **cả hai bên** so sánh?
5. `providedIn: 'root'` nghĩa là gì? Nếu không có nó thì chuyện gì xảy ra khi hai component cùng dùng một service?
6. `asReadonly()` dùng để làm gì? Vì sao service nên giữ signal gốc `private` và chỉ cho ra ngoài bản chỉ đọc?
7. Kể ra **ba nơi khác nhau** một thứ có thể "đi vào" component: `inject()`, mảng `imports`, và import thường. Cho một ví dụ cho mỗi nơi.
8. `inject()` gọi được ở đâu và **không** gọi được ở đâu? Lỗi gì xảy ra nếu gọi sai chỗ?
9. Vì sao `pathMatch: 'full'` là bắt buộc với route rỗng `path: ''`?
10. Phân biệt `href="/drinks/2"` và `[routerLink]="['/drinks', 2]"`. Hậu quả của việc dùng sai?
11. `route.params` và `route.paramMap` khác nhau thế nào? Cách lấy giá trị của mỗi loại?
12. Vì sao tham số lấy từ URL **luôn phải** đưa qua `Number()` trước khi so sánh với `id`?
13. So sánh Template-driven form (`ngModel`) và Reactive form (`formGroup`): mỗi loại phù hợp với tình huống nào?
14. Vì sao dùng `(ngSubmit)` trên thẻ `<form>` tốt hơn `(click)` trên nút gửi?
15. `.value` và `.getRawValue()` khác nhau ở kiểu trả về thế nào? Và `fb.nonNullable.group()` giải quyết thêm vấn đề gì?
16. Vì sao trong service **không được** dùng `.push()` để thêm phần tử vào mảng của signal?



PHẦN B — Tìm lỗi trong code

Mỗi đoạn có **đúng một lỗi**. Chỉ ra lỗi, giải thích **tại sao sai**, và viết lại cho đúng.

Lỗi số 1

```
<input [(ngModel)]="keyword" type="search" />
```

Lỗi số 2

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-drink-list',
  imports: [],
  templateUrl: './drink-list.html',
  styleUrls: ['./drink-list.css'],
})
export class DrinkList {
  protected readonly keyword = signal<string>('');
}
```

```
<input [ngModel]="keyword()" (ngModelChange)="keyword.set($event)" />
```

Lỗi số 3

```
protected readonly filteredDrinks = computed(() => {
  const key = this.keyword().toLowerCase().trim();
  return this.drinks().filter((drink) => drink.name.includes(key));
});
```

Lỗi số 4

```
import { Injectable, signal } from '@angular/core';
import { MOCK_DRINKS } from './mock-drinks';

@Injectable()
export class DrinkService {
  private readonly drinksState = signal(MOCK_DRINKS);
  readonly drinks = this.drinksState.asReadonly();
}
```

Lỗi số 5

```
export class DrinkList {
  protected readonly selectedDrink = signal<DrinkModel>
    (this.drinkService.drinks()[0]);

  private readonly drinkService = inject(DrinkService);
}
```

Lỗi số 6

```
export class DrinkDetail {
  protected goBack(): void {
```

```
const router = inject(Router);
router.navigate(['/drinks']);
}
}
```

Lỗi số 7

```
export const routes: Routes = [
  { path: 'drinks', component: DrinkList },
  { path: 'drinks/:id', component: DrinkDetail },
  { path: 'drinks/new', component: AddDrink },
  { path: '', redirectTo: 'drinks', pathMatch: 'full' },
];
```

Lỗi số 8

```
private readonly params = toSignal(this.route.params);

protected readonly selectedDrink = computed(() => {
  const id = Number(this.params()?.get('id'));
  return this.drinkService.getDrinkById(id);
});
```

Lỗi số 9

```
protected readonly selectedDrink = computed(() => {
  const id = this.params()?.get('id');
  return this.drinkService.getDrinkById(id);
});
```

Lỗi số 10

```
addDrink(newDrink: DrinkModel): void {
  this.drinksState().push(newDrink);
}
```

Lỗi số 11

```
@if (selectedDrink(); as drink) {
  <h2>{{ selectedDrink().name }}</h2>
}
```

Lỗi số 12

```
protected readonly drinkForm = this.fb.group({
  name: ['', Validators.required],
});

private readonly fb = inject(FormBuilder);
```

PHẦN C — Project: Quầy Trà Sữa 🥤

Bốn nhiệm vụ, làm tuần tự.

Nhiệm vụ C13 — ôn Bài 13: Two-Way Binding

Yêu cầu: Thêm ô tìm kiếm phía trên danh sách nút chọn món, có nhãn `<label>` gắn đúng cách bằng cặp `for / id`. Component ghi nhớ từ khóa đang gõ dưới dạng state, cập nhật theo từng ký tự.

Danh sách nút **không được lấy trực tiếp** từ danh sách gốc nữa, mà từ một dữ liệu dẫn xuất đã sàng lọc theo từ khóa. Việc tìm kiếm phải:

- Không phân biệt chữ hoa / chữ thường
- Bỏ qua khoảng trắng thừa hai đầu
- Khi ô trống thì hiện lại **tất cả** món

Khởi `@empty` giờ sẽ thật sự được dùng khi không món nào khớp.

🌀 **Tự kiểm tra:** gõ `mat` → chỉ còn Matcha. Gõ `TRÂN CHÂU` chữ hoa → vẫn ra món 1. Gõ `xyz` → hiện thông báo không tìm thấy. Xóa hết → cả 3 món trở lại.

🎨 Yêu cầu CSS cho C13

- **Ô tìm kiếm** — chiếm hết chiều ngang khung chứa, có `padding` bên trong để chữ không dán sát viền, viền bo góc mềm. Nhớ `box-sizing: border-box` kéo `width: 100%` cộng thêm padding làm ô tràn ra ngoài.
- **Nhãn `<label>`** — nhỏ và nhợt hơn nội dung chính, nhưng vẫn đủ tương phản để đọc được (tối thiểu 4.5:1 theo WCAG AA).
- **Danh sách nút chọn món** — dùng **Flexbox** để các nút nằm cùng hàng, tự xuống dòng khi màn hình hẹp, cách nhau đều nhau bằng `gap`. Bỏ dấu chấm đầu dòng của `<ul>`.
- **Nút đang được chọn** — phải **nhìn là biết ngay** khác các nút còn lại (đổi màu nền và màu chữ, không chỉ đổi viền mờ mờ).
- **Dòng "không tìm thấy"** — chữ nhợt hơn, có khoảng đệm, không dán sát mép.

Nhiệm vụ C14 — ôn Bài 14: Services & DI

```
ng generate service drink-service
```

Yêu cầu: `DrinkService` phải là **nơi duy nhất** trong dự án biết tới file `mock-drinks.ts`. Nó giữ danh sách bên trong ở dạng `private`, và chỉ cho bên ngoài **đọc** qua một bản chỉ đọc.

`DrinkList` phải **thôi import `MOCK_DRINKS`** hoàn toàn — tiêm service vào và lấy dữ liệu từ đó. Cả danh sách gốc lẫn món được chọn ban đầu đều lấy qua service.

Tự xét xem template có gọi trực tiếp tới service không, rồi quyết định **private** hay **protected**.

🔍 **Tự kiểm tra:** chạy `grep -rn "mock-drinks" src/` — kết quả phải chỉ còn **đúng một dòng**, và đó là dòng import trong file service.

🎨 Yêu cầu CSS cho C14

Không có. Service không có giao diện — đây là nhiệm vụ duy nhất trong bộ này được nghỉ tay về CSS. 😊

Nhiệm vụ C15 — ôn Bài 15: Routing

Yêu cầu: Biến ứng dụng một trang thành hai trang có URL riêng.

Bảng route cần: một route cho trang danh sách ở `/drinks`, một route cho trang chi tiết có tham số động nhận id, và một route chuyển hướng từ trang chủ về trang danh sách.

Component App trở thành khung chứa — chỗ đang gọi `<app-drink-list>` phải thay bằng cái khung để Router tráo component vào. Nhớ khai báo directive tương ứng, và **xóa** import `DrinkList` không còn dùng.

DrinkList thôi hiển thị phần chi tiết. Các nút chọn món đổi thành **liên kết điều hướng** tới trang chi tiết từng món — dùng đúng loại thẻ HTML dành cho việc đi tới nơi khác. Signal ghi nhớ món đang chọn và phương thức chọn món trở thành vô dụng, hãy dọn đi.

DrinkService cần thêm phương thức tra cứu món theo id.

DrinkDetail không còn nhận dữ liệu từ cha nên **gỡ bỏ cổng input**. Nó tự đọc id từ URL rồi hỏi service. Vì việc tra cứu có thể **không tìm thấy gì**, template phải có nhánh dự phòng. Thêm liên kết quay về trang danh sách, không có thì người dùng bị kẹt.

🔍 **Tự kiểm tra:** vào `/` → tự nhảy sang `/drinks`. Bấm một món → URL thành `/drinks/2`. Bấm nút Back của trình duyệt → về đúng trang trước. Nhấn F5 ở `/drinks/2` → vẫn ở món đó. Gõ tay `/drinks/999` → hiện thông báo không tìm thấy, **không** trắng trang.

🎨 Yêu cầu CSS cho C15

Đây là nhiệm vụ CSS nhiều nhất, vì nút bấm biến thành liên kết và thẻ `<a>` có mặc định rất xấu.

- **Nút chọn món giờ là `<a>`** — thẻ này mặc định có **gạch chân màu xanh tím**. Bạn phải `text-decoration: none` và đặt lại màu, để nó trông như nút chứ không như link Wikipedia.
- **Con trỏ chuột** — thẻ `<a>` không có `href` thường không hiện bàn tay khi hover. Kiểm tra và thêm `cursor: pointer` nếu cần.
- **Trạng thái hover** — liên kết phải có phản hồi khi trỏ chuột vào (đổi màu nền, hoặc nhấc nhẹ lên bằng `transform: translateY(-2px)`, hoặc thêm bóng).
- **Thẻ chi tiết món** — dùng **Flexbox** xếp dọc: ảnh trên, tên món, mô tả, bộ đếm số ly, tổng tiền, danh sách topping. Cách nhau bằng `gap`, có `padding` quanh viền.
- **Ảnh món** — giới hạn kích thước và dùng `object-fit: cover` để ảnh không bị méo.
- **Cụm `- / số ly / +`** — **Flexbox ngang**, căn giữa theo trục dọc (`align-items: center`), cách nhau đều. Nút ở trạng thái `:disabled` phải nhìn ra là đang bị khóa.
- **Thứ bậc thị giác** — tên món to và đậm nhất, mô tả nhỏ và nhạt hơn, tổng tiền nổi bật (đây là con số người dùng quan tâm nhất).
- **Liên kết quay lại** — nhỏ, đặt phía trên nội dung, có gạch chân khi hover.
- **Thông báo không tìm thấy** — căn giữa, có khoảng đệm rộng, đóng khung nhẹ để trông có chủ ý chứ không như trang bị lỗi.

Nhiệm vụ C16 — ôn Bài 16: Reactive Forms

ng generate component add-drink

Yêu cầu: Form kiểu Reactive với **ba ô** — tên món, mô tả, và giá cơ bản — cả ba bắt buộc điền. Mỗi ô có nhãn gắn đúng cách.

Khi gửi form: kiểm tra hợp lệ trước, dựng một **DrinkModel** hoàn chỉnh từ dữ liệu form cộng giá trị mặc định cho các trường form không hỏi tới, đưa vào service, rồi **tự động điều hướng** về trang danh sách.

DrinkService cần thêm phương thức nhận món mới. Nhớ nguyên tắc signal — tạo mảng mới.

Bảng route cần thêm route cho trang mới, và **đặt đúng thứ tự**.

DrinkList cần một liên kết dẫn tới trang thêm mới.

Về id: mỗi món cần id không trùng. Gợi ý: lấy id lớn nhất hiện có rồi cộng 1.

Về ảnh: form không hỏi ảnh, dùng tạm một địa chỉ mặc định.

Về giá: ô giá là số, nên `<input type="number">`. Chú ý kiểu dữ liệu form trả về.

🔗 **Tự kiểm tra:** bấm Lưu khi ô còn trống → không có gì xảy ra. Điền đủ → nhảy về `/drinks` và **thấy món mới trong danh sách**. Bấm vào món mới → mở được trang chi tiết với URL riêng. Tìm kiếm tên món mới → ra kết quả.

🎨 Yêu cầu CSS cho C16

- **Cả form** — dùng **Flexbox xếp dọc** với **gap** giữa các ô. Đóng khung có **padding** để form trông như một khối liền mạch chứ không phải mấy ô lạc trôi.
- **Từng cặp nhãn + ô nhập** — cũng là một Flexbox nhỏ xếp dọc, **gap** khoảng `0.375rem`. Nhớ bài học **Flexbox lồng nhau** ở Bài 10.
- **Ba ô nhập** — cùng kích thước, cùng kiểu viền, cùng bo góc. Sự **nhất quán** quan trọng hơn đẹp: ba ô lệch nhau một chút là nhìn ra ngay.
- **font-family: inherit** cho `<input>` và `<button>` — nếu thiếu, trình duyệt dùng phong mặc định của hệ thống và chúng sẽ **lạc phong** so với phần còn lại của trang. Đây là chi tiết rất nhiều người bỏ sót.
- **Nút Lưu** — màu nền đậm nổi bật vì đây là **hành động chính** của trang. Có `:hover`. Đứng cho nó chiếm hết chiều ngang (`align-self: flex-start` giúp nó chỉ rộng bằng nội dung).
- **Ô đang được focus** — kiểm tra xem viền focus có thấy rõ không. Nếu bạn ghi đề **outline** thì **phải thay bằng dấu hiệu khác** (viền đậm hơn, đổ bóng). Người dùng bàn phím cần biết mình đang ở ô nào — đây là yêu cầu WCAG, không phải tùy chọn.

Nhiệm vụ C-CSS — ôn Bài 10: tổng kết trang trí

Sau khi xong C13–C16, hãy nhìn lại **toàn bộ ứng dụng** một lượt và đánh giá theo **ba tiêu chí của Bài 10**:

Tiêu chí	Câu hỏi tự kiểm
Flexbox	Còn chỗ nào bố cục đang phụ thuộc vào bản tính mặc định của thẻ (block/inline) thay vì do bạn chủ động quyết định?
Thứ bậc thị giác	Nhìn vào mỗi trang trong 1 giây, mắt bạn có bị hút vào đúng thứ quan trọng nhất không?
Khoảng trắng	Có chỗ nào nội dung dán sát mép màn hình hoặc dính chặt vào nhau không?

Thêm hai việc mang tính toàn cục:

- **Khung ngoài** — đặt `max-width` (khoảng `640px–720px`) và `margin: 0 auto` cho khung chứa chính, để trên màn hình rộng nội dung không bị kéo dài thành một dòng chữ chạy hết màn hình. Gợi ý: dùng `:host` trong `app.css`.
- **Nền và phong toàn trang** — đặt `background-color`, `color`, `font-family` cho `body` trong `src/styles.css`. Đây là file CSS **toàn cục**, khác với CSS của component chỉ áp dụng cục bộ.

🎨 Bảng màu tặng kèm

Nếu bạn không muốn tự chọn màu, đây là bảng màu "bếp ấm" — dùng luôn cho khỏi mất thời gian. Tôi đã kiểm tra độ tương phản của các cặp chữ/nền đạt WCAG AA:

```
/* Nền */
--kem-nhat: #fffd7; /* nền thẻ, nền ô nhập */
--kem: #fdf6e9; /* nền trang */
--kem-dam: #f0e2c8; /* viền */

/* Nhấn (dùng cho nút chính, nút đang chọn) */
--cam-nhat: #fbe3d6;
--cam: #d2683a;
--cam-dam: #b8512a;
--cam-tham: #93401f; /* tiêu đề */

/* Phụ (dùng cho nhãn số lượng, huy hiệu) */
--xanh-nhat: #e3f0e0;
--xanh-dam: #38603b;

/* Chữ */
--chu-chinh: #3a332c; /* nội dung chính */
--chu-nhat: #6b6259; /* nhãn, mô tả, chữ thứ yếu */
```

💡 Bạn có thể khai báo chúng thành biến CSS thật trong `styles.css`:

```
:root {
  --cam: #d2683a;
  /* ... */
}
```

rồi dùng `background-color: var(--cam);`. Cách này giúp đổi màu cả app chỉ bằng sửa một chỗ — một kỹ thuật rất đáng học, dù không bắt buộc ở bài này.

🚀 PHẦN D — Thử thách nâng cao (không bắt buộc)

1. **Tìm kiếm theo nhiều trường**. Mở rộng bộ lọc để tìm cả trong `description`, không chỉ `name`.
2. **Sắp xếp theo giá**. Thêm hai nút "Giá tăng dần" / "Giá giảm dần". Gợi ý: một signal lưu kiểu sắp xếp, và một `computed` dùng `.sort()`. ⚠️ `.sort()` **sửa trực tiếp mảng gốc** — hãy tìm hiểu cách tạo bản sao trước khi sắp xếp (spread operator giúp được).

3. **Đếm kết quả tìm kiếm.** Hiện dòng "Tìm thấy N món" phía trên danh sách, dùng `computed`.
4. **Xóa món.** Thêm phương thức `deleteDrink(id)` vào service dùng `.filter()`, và một nút xóa ở trang chi tiết. Sau khi xóa thì điều hướng về danh sách.
5. **Đánh dấu yêu thích.** Thêm phương thức `toggleFavorite(id)` vào service dùng `.map()` để đảo giá trị `isPopular` của đúng một món.
6. **Route bắt lỗi 404.** Thêm route `{ path: '**', component: NotFound }` cho URL không tồn tại. Tìm hiểu xem vì sao nó **phải** đặt ở cuối cùng.

Bảng tự theo dõi tiến độ

Phần	Nội dung	Đã xong?
A	16 câu lý thuyết	☐
B	12 đoạn code tìm lỗi	☐
C13	Ô tìm kiếm + <code>computed</code> lọc	☐
C13 🌈	CSS: ô tìm kiếm + hàng nút Flexbox	☐
C14	<code>DrinkService</code> + <code>inject()</code>	☐
C15	Routing hai trang + <code>ActivatedRoute</code>	☐
C15 🌈	CSS: gỡ gạch chân <code><a></code> , thẻ chi tiết, hover	☐
C16	Reactive form thêm món	☐
C16 🌈	CSS: form, ô nhập nhất quán, nút chính	☐
C-CSS	Tổng kết: khung ngoài, nền toàn trang, 3 tiêu chí	☐
D1	Tìm cả trong mô tả	☐
D2	Sắp xếp theo giá	☐
D3	Đếm kết quả	☐
D4	Xóa món	☐
D5	Đảo trạng thái yêu thích	☐
D6	Route 404	☐

Sổ tay tra cứu nhanh (Bài 13 → 16)

★ Ba nơi một thứ có thể "đi vào" component

Đây là chỗ dễ lẫn nhất của Giai đoạn 4.

Cơ chế	Dành cho	Ví dụ
<code>inject()</code>	Service (thứ trong hệ thống DI)	<code>Router</code> , <code>ActivatedRoute</code> , <code>FormBuilder</code> , service của bạn

Cơ chế	Dành cho	Ví dụ
mảng imports	Component / Directive / Pipe / Module dùng trong template	RouterLink, RouterOutlet, FormsModule, ReactiveFormsModule, JsonPipe
import thường	Interface / hằng số / hàm	DrinkModel, MOCK_DRINKS, Validators, signal, computed, toSignal

Cách nhận biết trong 3 giây:

1. Nó có **@Injectable()** (hoặc là service có sẵn của Angular)? → **inject()**
2. Nó xuất hiện trong file **.html**? → mảng **imports**
3. Nó chỉ là kiểu dữ liệu / mảng dữ liệu / hàm? → import thường

inject() chỉ gọi được ở "ngữ cảnh tiêm" — dòng khai báo thuộc tính hoặc **constructor**. Gọi trong phương thức sẽ lỗi **NG0203**.

Two-way binding với signal

```
<label for="search">Tìm món</label>
<input
  id="search"
  type="search"
  [ngModel]="keyword()"
  (ngModelChange)="keyword.set($event)"
/>
```

```
import { FormsModule } from '@angular/forms';

@Component({ imports: [FormsModule] /* ... */ })
```

Cú pháp gộp **[(ngModel)]** không dùng được với **signal** vì nó cần gán trực tiếp, còn signal phải gọi **.set()**.

Ba phương thức mảng

Phương thức	Trả về	Dùng khi
.map()	Mảng mới đã biến đổi	Nhân số lượng theo số ly
.filter()	Mảng mới đã sàng lọc	Tìm kiếm, xóa phần tử
.find()	Một phần tử hoặc undefined	Tra cứu theo id

Tìm kiếm không phân biệt hoa thường:

```
const key = this.keyword().toLowerCase().trim();
if (key === '') {
  return all; // ô trống → trả tất cả
}
return all.filter((d) => d.name.toLowerCase().includes(key));
```

Service

```
@Injectable({
  providedIn: 'root', // ★ singleton – MỘT bản duy nhất cho cả app
})
export class DrinkService {
  private readonly drinksState = signal<DrinkModel[]>(MOCK_DRINKS); // gốc, chỉ
  service sửa
  readonly drinks = this.drinksState.asReadonly(); // bản chỉ
  đọc cho bên ngoài

  getDrinkById(id: number): DrinkModel | undefined {
    return this.drinksState().find((d) => d.id === id);
  }

  addDrink(newDrink: DrinkModel): void {
    this.drinksState.update((current) => [...current, newDrink]); // ⚠ KHÔNG
    .push()
  }
}
```

Trong component:

```
private readonly drinkService = inject(DrinkService); // ⚠ phải khai báo TRƯỚC
dòng dùng nó
protected readonly selected = signal(this.drinkService.drinks()[0]);
```

Routing

```
export const routes: Routes = [
  { path: 'drinks', component: DrinkList },
  { path: 'drinks/new', component: AddDrink }, // ★ CỤ THỂ đặt TRƯỚC
  { path: 'drinks/:id', component: DrinkDetail }, // ★ ĐỘNG đặt SAU
  { path: '', redirectTo: 'drinks', pathMatch: 'full' },
];
```

Router lấy route khớp ĐẦU TIÊN từ trên xuống. Đặt `drinks/:id` trước `drinks/new` thì `/drinks/new` sẽ bị hiểu thành `id = "new"`.

```
<!-- Khung tráo component – mọi thứ ngoài nó giữ nguyên khi chuyển trang -->
<router-outlet />

<!-- Điều hướng: dùng <a> chứ không <button>, dùng routerLink chứ không href -->
<a [routerLink]="['/drinks', drink.id]">{{ drink.name }}</a>
<a routerLink="/drinks">Quay lại</a>
```

Đọc tham số URL:

```
private readonly route = inject(ActivatedRoute);
private readonly params = toSignal(this.route.paramMap); // ⚠️ paramMap, KHÔNG phải params

protected readonly selectedDrink = computed(() => {
  const id = Number(this.params()?.get('id')); // ⚠️ nhớ Number()
  return this.drinkService.getDrinkById(id);
});
```

Thuộc tính	Là gì	Lấy giá trị
<code>route.params</code>	Đối tượng thường <code>{id: '1'}</code>	<code>params()?.['id']</code>
<code>route.paramMap</code>	Đối tượng có <code>.get()</code>	<code>paramMap()?.get('id')</code>

Xử lý "không tìm thấy" — **bắt buộc dùng `as`**:

```
@if (selectedDrink(); as drink) {
  <h2>{{ drink.name }}</h2> <!-- ⚠️ dùng `drink`, KHÔNG dùng selectedDrink() -->
} @else {
  <p>Không tìm thấy món này.</p>
}
```

TypeScript **không thu hẹp kiểu cho lời gọi hàm** — mỗi lần gọi `selectedDrink()` là một lần chạy mới, nên nó không dám bảo đảm kết quả. `as` chốt giá trị vào một biến ổn định.

Điều hướng bằng code:

```
private readonly router = inject(Router);
this.router.navigate(['/drinks']);
```

Reactive Forms

```
private readonly fb = inject(FormBuilder); // ⚠️ khai báo TRƯỚC form

protected readonly drinkForm = this.fb.nonNullable.group({
  name: ['', Validators.required],
  giaCoBan: [0, Validators.required],
});

protected save(): void {
  if (this.drinkForm.invalid) {
    this.drinkForm.markAllAsTouched();
    return;
  }
  const { name, giaCoBan } = this.drinkForm.getRawValue();
  // ...
}
```

```
<form [formGroup]="drinkForm" (ngSubmit)="save()">
  <label for="name">Tên món</label>
  <input id="name" type="text" FormControlName="name" />

  <button type="submit">Lưu</button>
</form>
```

```
import { ReactiveFormsModule } from '@angular/forms';

@Component({ imports: [ReactiveFormsModule] /* ... */ })
```

Bộ đôi **nonNullable** + **getRawValue()**:

Cách viết	Kiểu trả về	Phải viết ?
fb.group + .value	string undefined	Có
fb.group + .getRawValue()	string null	Có
fb.nonNullable.group + .getRawValue()	string	Không

- **.value** bỏ qua ô **disabled** → mọi trường thành tùy chọn (**undefined**)
- **FormControl nullable mặc định** vì **reset()** đưa về **null**
- **nonNullable** khử **null**, và làm **reset()** trả về **giá trị ban đầu** thay vì **null**

CSS — nhắc nhanh (Bài 10)

CSS của component **chỉ áp dụng cho template của chính nó**, không rò rỉ ra ngoài. CSS toàn cục thì viết trong **src/styles.css**.

```
/* Nhắm vào chính thẻ bọc ngoài của component (vd <app-drink-list>) */
:host {
  display: block;
  max-width: 640px;
  margin: 0 auto;
  padding: 2rem 1.25rem;
}

/* Flexbox xếp DỌC – các khối nội dung */
.card {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding: 1.5rem;
}

/* Flexbox xếp NGANG – cụm nút, hàng có 2 đầu */
.row {
  display: flex;
  align-items: center; /* căn giữa theo trục dọc */
  justify-content: space-between; /* đẩy 2 đầu ra xa */
  gap: 0.75rem;
}
```

```

    flex-wrap: wrap;                /* tự xuống dòng khi hẹp */
  }

  /* Thứ bậc thị giác: to hơn + đậm hơn + đậm màu hơn = quan trọng hơn */
  .title {
    font-size: 1.75rem;
    font-weight: 700;
    color: #93401f;
  }
  .desc {
    font-size: 1rem;
    line-height: 1.6;
    color: #6b6259;
  }

  /* Định dạng có điều kiện – dùng [class.x], KHÔNG dùng ngClass */
  .is-active {
    color: #ffffff;
    background-color: #d2683a;
  }

```

```
<button [class.is-active]="selectedDrink().id === drink.id">...</button>
```

Những cái bẫy CSS hay dính

Bẫy	Hậu quả	Cách khử
<code>width: 100%</code> mà quên <code>box-sizing</code>	Ô nhập tràn ra ngoài khung	<code>box-sizing: border-box</code>
<code><a></code> dùng làm nút	Có gạch chân, màu xanh tím	<code>text-decoration: none</code> + đặt lại <code>color</code>
<code><input></code> / <code><button></code> thiếu <code>font-family</code>	Lạc phong so với cả trang	<code>font-family: inherit</code>
Ghi đè <code>outline</code> mà không thay thế	Người dùng bàn phím không biết đang ở đâu	Giữ <code>:focus-visible</code> hoặc thay bằng viền/bóng rõ
<code>list-style: none</code> trên <code><div></code>	Vô tác dụng — chỉ có hiệu lực trên <code>/</code>	Dùng <code></code> , hoặc bỏ dòng CSS đó
Chữ quá nhạt cho "đẹp"	Không đạt tương phản 4.5:1, người mắt kém không đọc được	Nhạt nhất nên dừng ở khoảng <code>#6b6259</code>
Dùng <code>px</code> cho cỡ chữ	Không co giãn khi người dùng phóng to chữ	Dùng <code>rem</code>
CSS chết (class không ai dùng)	Người đọc sau bối rối	Xóa, hoặc gắn class vào HTML

Những cái bẫy hay dính

Bẫy	Sai	Đúng
Cú pháp gộp với signal	<code>[(ngModel)]="keyword"</code>	<code>[ngModel] + (ngModelChange)</code>
Quên <code>FormsModule</code> / <code>ReactiveFormsModule</code>	<code>imports: []</code>	thêm module tương ứng
Lọc rồi nhưng template vẫn lặp mảng gốc	<code>@for (d of drinks())</code>	<code>@for (d of filteredDrinks())</code>
Service thiếu <code>providedIn</code>	<code>@Injectable()</code>	<code>@Injectable({ providedIn: 'root' })</code>
Sai thứ tự khai báo	dùng <code>this.x</code> trước dòng <code>inject</code>	<code>inject()</code> khai báo lên trên
<code>inject()</code> trong phương thức	<code>save() { inject(Router) }</code>	tiêm ở dòng khai báo thuộc tính
Route động đặt trước route cụ thể	<code>:id</code> trên <code>new</code>	<code>new</code> trên <code>:id</code>
Thiếu <code>pathMatch: 'full'</code>	<code>{ path: '', redirectTo: 'x' }</code>	thêm <code>pathMatch: 'full'</code>
<code>params</code> mà gọi <code>.get()</code>	<code>route.params + .get()</code>	<code>route.paramMap + .get()</code>
Quên đổi kiểu tham số URL	<code>getId(params()?.get('id'))</code>	<code>getId(Number(params()?.get('id')))</code>
<code>href</code> thay <code>routerLink</code>	<code>href="/drinks/2"</code>	<code>[routerLink]="['/drinks', 2]"</code>
Còn gọi hàm trong khối <code>as</code>	<code>{{ selectedDrink().name }}</code>	<code>{{ drink.name }}</code>
<code>.push()</code> vào signal	<code>drinksState().push(x)</code>	<code>drinksState.update(c => [...c, x])</code>
<code>(click)</code> trên nút gửi form	<code><button (click)="save()"></code>	<code><form (ngSubmit)="save()"></code>
Quên xóa import không dùng	để lại → NG8113	xóa khỏi cả <code>import</code> và mảng <code>imports</code>